```
"""fsm.py — Finite State Machine infrastructure for [illegible].

[illegible] Pygame standard: game flow is governed by a strict FSM with
`enter()/exit()/update(dt)/render()` per state, instead of loose string
variables and boolean flags.

This module provides the base classes. The actual State subclasses live in
`[illegible].py` because they need access to module-level state (mouse coords,
menu buttons, save slots, etc.) that's defined there.

Public API:
    BaseState          — base class. Override enter/exit/handle_event/update/
                         render/set_ambience.
    GameStateManager   — registry + current state + transition machinery.
                         `transition(name)` is instant; `trigger_transition(name)`
                         does a fade-out → swap → fade-in.

Backwards compat:
    The manager exposes `current_name` and `previous_name` strings so legacy
    code that did `if game_state == "MENU":` can be migrated incrementally to
    `if gsm.current_name == "MENU":` and then fully to FSM dispatch.
"""
from __future__ import annotations

import pygame

class BaseState:
    """Base class for a single game state (MENU, GAME, SETTINGS, etc.).

    Subclasses override these methods. The default implementations are no-ops
    so a subclass only needs to implement what it cares about.
    """
    name: str = "BASE"

    def enter(self) -> None:
        """Called once when this state becomes active."""
        pass

    def exit(self) -> None:
        """Called once when this state is being left."""
        pass

    def handle_event(self, event: pygame.event.Event,
                     mouse_x: int, mouse_y: int,
                     keys: pygame.key.ScancodeWrapper) -> bool:
        """Handle a single SDL event.

        Returns True if the event was consumed (no further processing).
        Returning False (the default) lets global handlers see it too.
        """
        return False

    def update(self, dt: float) -> None:
        """Advance this state's simulation by `dt` seconds."""
        pass

    def set_ambience(self) -> None:
        """Set audio ambience for this state. Called once per frame."""
        pass

    def render(self, surface: pygame.Surface) -> None:
        """Draw this state to `surface`."""
        pass

class GameStateManager:
    """Registry + current-state + fade-transition machinery.

    Usage:
        gsm = GameStateManager()
        gsm.add(MenuState())
        gsm.transition("MENU")
        # in main loop:
        gsm.current.handle_event(event, mx, my, keys)
        gsm.current.update(dt)
        gsm.current.render(screen)
        gsm.update_transition()
        gsm.draw_transition(screen)
    """
    def __init__(self) -> None:
        self.states: dict[str, BaseState] = {}
        self.current: BaseState | None = None
        self.current_name: str = ""
        self.previous_name: str = ""
        # Fade transition state. `trigger_transition(name)` starts a fade-out;
        # when fade_alpha reaches 255, the manager swaps to the new state and
        # reverses the fade (fade_speed < 0). When fade_alpha returns to 0,
        # transitioning becomes False.
        self.transitioning: bool = False
        self.next_state: str = ""
        self.fade_alpha: int = 0
        self.fade_speed: int = 0
        # Reusable fade surface (lazy-allocated on first draw_transition).
        self._fade_surf: pygame.Surface | None = None

    def add(self, state: BaseState) -> None:
        if self.states.get(state.name) is not None:
            raise ValueError(f"State {state.name!r} already registered")
        self.states[state.name] = state

    def transition(self, name: str) -> None:
        """Instant transition: exit current, enter new. No fade."""
        if name not in self.states:
            raise KeyError(f"Unknown state {name!r}; registered: {list(self.states)}")
        if self.current is not None:
            self.current.exit()
        self.previous_name = self.current_name
        self.current_name = name
        self.current = self.states[name]
        self.current.enter()

    def trigger_transition(self, name: str) -> None:
        """Faded transition: schedule a fade-out → swap → fade-in.

        Sets `transitioning=True` and `next_state=name`. The main loop must
        call `update_transition()` each frame to advance the fade, and
        `draw_transition(screen)` to blit the fade overlay.
        """
        if name not in self.states:
            raise KeyError(f"Unknown state {name!r}; registered: {list(self.states)}")
        self.transitioning = True
        self.next_state = name
        self.fade_speed = 10  # fade-out speed (alpha units per frame)
        # fade_alpha starts at 0 (transparent) and increases toward 255.

    def update_transition(self) -> None:
        """Advance the fade transition. Swaps state at peak alpha.

        Call this every frame regardless of `transitioning` — it's a no-op
        when no transition is in progress.
        """
        if not self.transitioning:
            return
        self.fade_alpha += self.fade_speed
        if self.fade_alpha >= 255:
            # Peak: swap state, reverse fade direction.
            self.fade_alpha = 255
            self.fade_speed = -10  # fade-in speed (negative = alpha decreases)
            # Perform the actual state swap.
            if self.current is not None:
                self.current.exit()
            self.previous_name = self.current_name
            self.current_name = self.next_state
            self.current = self.states[self.next_state]
            self.current.enter()
        elif self.fade_alpha <= 0 and self.fade_speed < 0:
            # Fade-in complete.
            self.fade_alpha = 0
            self.transitioning = False

    def draw_transition(self, surface: pygame.Surface) -> None:
        """Draw the fade overlay if a transition is in progress."""
        if not self.transitioning:
            return
        if self._fade_surf is None or self._fade_surf.get_size() != surface.get_size():
            self._fade_surf = pygame.Surface(surface.get_size())
        self._fade_surf.fill((0, 0, 0))
        self._fade_surf.set_alpha(min(255, max(0, self.fade_alpha)))
        surface.blit(self._fade_surf, (0, 0))
```